

Mejor ranking y primer descubrimiento: BM25 + K-Means

Arreglar lo que TF-IDF rankeaba mal, medirlo con rigor, y dejar que el corpus se agrupe solo

Ramsés Alejandro Camas Nájera

Universidad Politécnica de Chiapas · 9º cuatrimestre · Semana 5, Sesión 2 · Junio 2026

La sesión pasada construyeron — y rompieron — un buscador

Lo que lograron

- Pipeline de preprocesamiento en español.
- TF-IDF y similitud coseno desde cero.
- Un motor de búsqueda con ranking por relevancia.

Lo que NO funcionó

- “problemas de agua” no encontró “crisis hídrica”.
- Documentos largos parecían siempre más relevantes.
- Un término repetido 20 veces dominaba el score.

Dos tipos de falla, muy distintos

Fallas de RANKING (sesgo por longitud, saturación de términos) → las arregla BM25, hoy.

Fallas de SIGNIFICADO (agua $\approx$ hídrico) → ninguna reponderación las arregla. Necesitan embeddings (Semana 7).

Agenda — cerrar el ranking, medirlo, y descubrir



Bloque 1 — De TF-IDF a BM25

Saturación de TF, normalización por longitud, la fórmula y su implementación. Comparativa de rankings.

~60 min



Bloque 2 — ¿Cómo sabemos que es bueno?

Juicios de relevancia y métricas de IR: precision@k, recall, MRR, MAP, nDCG. TF-IDF vs. BM25 con números.

~50 min



Bloque 3 — Descubrir sin consulta

K-Means sobre los vectores TF-IDF: el mismo algoritmo de la Unidad 1, ahora sobre texto.

~50 min



Cierre — El puente

¿“crisis hídrica” y “agua potable” caen en el mismo grupo? La señal que falta.

~15 min

01

De TF-IDF a BM25

TF-IDF rankea, pero rankea con dos sesgos. BM25 es la corrección que Elasticsearch usa en producción.

- Por qué TF-IDF rankea mal
- Saturación de TF (k_1)
- Normalización por longitud (b)
- La fórmula, el código y la comparativa

Los dos sesgos de ranking de TF-IDF



1 • TF crece sin límite

Si “café” aparece 2 veces, el documento trata de café. ¿Y si aparece 40 veces? No es $20\times$ más relevante — pero TF-IDF lo trata así.

La primera mención importa mucho; la mención número 30, casi nada. TF-IDF no modela esos rendimientos decrecientes.



2 • Los documentos largos ganan

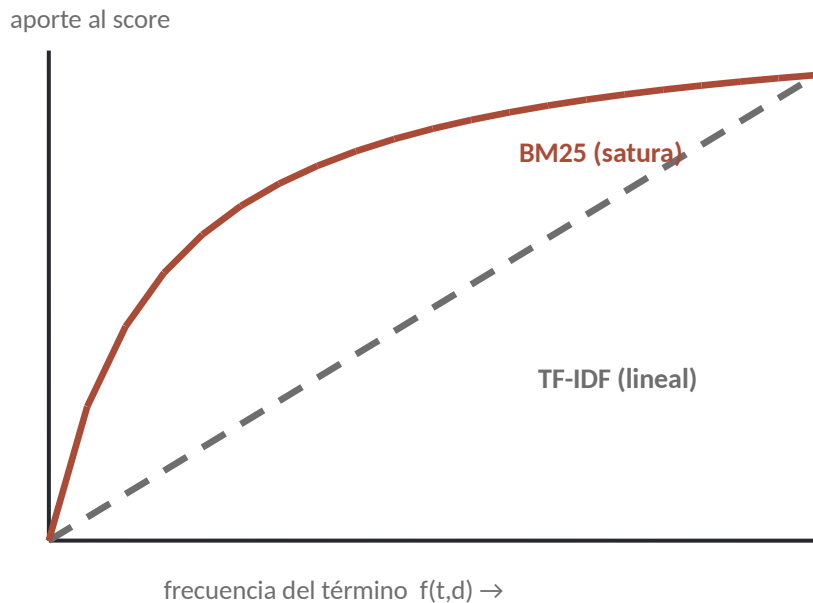
Un documento largo acumula más ocurrencias de cualquier término solo por ser largo, no por ser más relevante.

La normalización por $|d|$ de TF-IDF ayuda, pero es tosca: penaliza igual a un documento bien enfocado que a uno inflado con relleno.

BM25 ataca exactamente estos dos sesgos — con un parámetro para cada uno: k_1 y b .

Idea 1: saturación de la frecuencia (k_1)

BM25 hace que el aporte de un término se aplane: las primeras menciones suman mucho, las siguientes cada vez menos.



El componente de TF en BM25

$$\frac{f \cdot (k_1 + 1)}{f + k_1}$$

El parámetro k_1

Controla qué tan rápido satura. k_1 pequeño = satura pronto (casi binario); k_1 grande = más parecido a TF lineal. **Valor típico: 1.2–2.0.**

Idea 2: normalización por longitud (b)

BM25 penaliza a un documento por ser más largo que el promedio — pero de forma ajustable, no a ciegas.

El factor de longitud en el denominador

$$1 - b + b \cdot (|d| / \text{avgdl})$$

$|d|$ = longitud del documento · avgdl = longitud media del corpus

La intuición

Una mención de “sequía” en un tuit de 15 palabras dice más que la misma mención perdida en un informe de 5,000. BM25 descuenta al documento largo según cuánto excede el promedio del corpus.

$$b = 0$$

Longitud ignorada. El factor es 1: BM25 no penaliza documentos largos.

$$b = 1$$

Penalización completa, proporcional a $|d|/\text{avgdl}$. Castiga fuerte la longitud.

$$b = 0.75$$

El punto medio empírico que usa casi todo el mundo (Lucene, Elasticsearch).

BM25: las dos ideas en una fórmula

$$\text{score}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f \cdot (k_1 + 1)}{f + k_1 \cdot (1 - b + b \cdot |d|/\text{avgdl})}$$

IDF(t)

Rareza global — la misma idea de la sesión pasada. BM25 usa una variante suavizada que nunca es negativa.

Numerador (k_1)

Satura la frecuencia: las menciones extra valen cada vez menos. Techo en (k_1+1) .

Denominador (b)

Normaliza por longitud contra el promedio del corpus. $b=0.75$ por defecto.

Si $b=0$ y $k_1 \rightarrow \infty$, BM25 se reduce esencialmente a TF-IDF. Es una generalización, no un algoritmo ajeno.

El efecto: cómo cambia el ranking

Consulta “café chiapas” sobre el corpus chiapaneco. Mismo corpus, mismo preprocesamiento — distinto ranking.

Documento	d	Rank TF-IDF	Rank BM25	Qué pasó
d03 · Café rompe récord de exportación	corto	2	1	<i>Sube: foco, sin relleno</i>
d12 · Feria celebra el café y el cacao	medio	1	2	<i>Baja: “café” repetido ya no domina</i>
d09 · San Cristóbal, destino cultural	largo	3	4	<i>Baja: penalizado por longitud</i>
d08 · Repunta la producción de cacao	medio	4	3	<i>Sube: relevante, antes opacado</i>

El patrón: BM25 premia el documento enfocado y corto sobre el largo o el que solo repite el término. Eso es justo lo que un humano llamaría “más relevante”.

BM25 desde cero — reutilizando su TF-IDF

```
import math
from collections import Counter

def idf_bm25(corpus):
    N = len(corpus)
    df = Counter(t for d in corpus
                  for t in set(d))
    return {t: math.log(1 + (N-df[t]+.5)/(df[t]+.5))
            for t in df}

def bm25(doc, q, IDF, avgdl, k1=1.5, b=0.75):
    f = Counter(doc); dl = len(doc); s = 0.0
    for t in q:
        if t not in f: continue
        num = f[t] * (k1 + 1)
        den = f[t] + k1*(1 - b + b*dl/avgdl)
        s += IDF.get(t, 0) * num/den
    return s
```

Para notar

- Reutiliza el conteo de términos que ya tenían en TF-IDF.
- avgdl se calcula una vez sobre todo el corpus.
- k_1 y b son argumentos: pueden barrerlos y ver el efecto.
- No hay vectores ni coseno: BM25 acumula score por término.

Lab 3 — BM25 y comparación de rankings



Sobre el corpus y el pipeline del Lab 1. Reutilicen su motor TF-IDF del Lab 2 como punto de comparación.

1

Implementar BM25 — `idf_bm25` y `bm25` desde cero; calcular `avgdl` una sola vez sobre el corpus.

2

Buscar con BM25 — función `buscar_bm25(consulta, k=5)` análoga a la del Lab 2, pero ranqueando por score BM25.

3

Comparar lado a lado — para 3 consultas, mostrar el top-5 de TF-IDF y de BM25 en paralelo; marcar qué cambió de posición.

4

Barrer k_1 y b — probar $(k_1, b) \in \{1.2, 2.0\} \times \{0, 0.75, 1\}$; observar y documentar cómo se mueve el ranking.

Pregunta para la defensa: ¿qué combinación de k_1 y b dio el mejor ranking... y cómo lo saben sin medirlo? (Bloque 2)

02

¿Cómo sabemos que es bueno?

Ajustaron BM25 a ojo. Para afirmar que un buscador es mejor que otro hace falta medir — con números, no con intuición.

- Juicios de relevancia (qrels)
- Precision@k y Recall
- MRR y MAP
- nDCG: relevancia graduada

“Mi buscador funciona” no es una medición

Para comparar TF-IDF vs. BM25 (o ajustar k_1 , b) necesitan una pregunta responsable: **dada una consulta, ¿qué documentos son realmente relevantes, y qué tan arriba los puso el sistema?**



Verdad de referencia

Un conjunto de juicios: para cada consulta, qué documentos son relevantes. Sin esto no hay nada que medir.



Precisión vs. exhaustividad

¿Lo que devuelvo es relevante?
¿Encontré todo lo relevante? Casi
siempre hay que equilibrar ambas.



El orden importa

Acertar en la posición 1 vale más que en la 10. Las métricas buenas premian el buen ranking, no solo el acierto.

Mismo espíritu que su investigación: una afirmación sin métrica ni baseline no es evidencia.

La verdad de referencia: juicios de relevancia

Para un conjunto de consultas, alguien (ustedes) etiqueta qué documentos son relevantes. Binario (sí/no) o graduado (0–3).

Consulta	Relevantes (binario)	Graduado (0–3)
<i>sequía y cultivos</i>	d04, d02	d04=3, d02=2, d13=1
<i>café y exportación</i>	d03, d12, d08	d03=3, d12=2, d08=2
<i>problemas de agua</i>	d13, d02	d13=3, d02=3, d04=1

Por qué es lo más caro de IR

Etiquetar a mano no escala: un corpus web tiene millones de documentos. En la práctica se usa pooling (solo se juzga el top-k de varios sistemas).

Cuidado con el sesgo

Quien etiqueta define qué es “relevante”. d02 es relevante para “problemas de agua” aunque no contenga la palabra “agua” — el juicio humano sí ve el significado.

Precision@k y Recall

Precision@k

$$\frac{\text{relevantes en el top-k}}{k}$$

De lo que muestro arriba, ¿qué fracción sirve?

Recall

$$\frac{\text{relevantes recuperados}}{\text{total de relevantes}}$$

De todo lo relevante que existe, ¿cuánto encontré?

Ejemplo — consulta “café y exportación” (relevantes: d03, d12, d08)

El sistema devuelve, en orden: d03 ✓ d09 ✗ d12 ✓ d08 ✓ d05 ✗

Precision@3 = 2/3 = 0.67 (d03, d12 relevantes de los 3 primeros)

Recall@5 = 3/3 = 1.00 (encontró los 3 relevantes que existían)

MRR y MAP: métricas sensibles al orden

MRR — Mean Reciprocal Rank

Promedio de $1/(\text{posición del primer relevante})$.

1º relevante en pos. 1 $\rightarrow 1.0$
en pos. 2 $\rightarrow 0.5$
en pos. 4 $\rightarrow 0.25$

Ideal cuando el usuario solo quiere UN buen resultado (p. ej. “voy a la suerte”).

MAP — Mean Average Precision

Para cada consulta: promedia la precisión en cada posición donde aparece un relevante (Average Precision). Luego promedia entre consultas.

$$\begin{aligned} & d03 \checkmark \quad d09 \times \quad d12 \checkmark \quad d08 \checkmark \\ AP &= (1/1 + 2/3 + 3/4)/3 \\ &= 0.81 \end{aligned}$$

Resume precisión y recall en un solo número por consulta. El estándar histórico de IR.

nDCG: cuando la relevancia tiene grados

Las métricas anteriores son binarias (relevante o no). nDCG aprovecha relevancia graduada (0–3) y descuenta por posición.

DCG@k

$$\sum \text{rel}(i) / \log_2(i + 1)$$

Suma la ganancia de cada resultado, descontada por qué tan abajo está.

nDCG = DCG / IDCG

Se normaliza contra el orden ideal (IDCG): el ranking perfecto da nDCG = 1.0. Comparable entre consultas.

Mini ejemplo — relevancias devueltas en orden: 3, 1, 0, 2

$$\text{DCG} = 3/\log_2 2 + 1/\log_2 3 + 0/\log_2 4 + 2/\log_2 5 = 3 + 0.63 + 0 + 0.86 = 4.49$$

$$\text{IDCG} = \text{orden ideal } (3, 2, 1, 0) = 3 + 1.26 + 0.5 + 0 = 4.76$$

$$\text{nDCG} = 4.49 / 4.76 = 0.94$$

Ahora sí: TF-IDF vs. BM25, con números

Sobre un corpus realista (miles de documentos), el patrón típico es que BM25 mejora en las métricas sensibles al ranking:

Sistema	P@5	MRR	MAP	nDCG@5
<i>TF-IDF (Lab 2)</i>	0.53	0.72	0.61	0.78
<i>BM25 ($k_1=1.5, b=0.75$)</i>	0.60	0.83	0.69	0.86
Δ (mejora de BM25)	+0.07	+0.11	+0.08	+0.08

Pero cuidado en su Lab: sobre 14 documentos y 5 consultas, las diferencias caen dentro del ruido y BM25 puede empatar o perder. Evaluar con muestras chicas engaña — esa también es la lección.

Lab 3b — Midan su propio buscador



Construyan los qrels de su corpus y conviertan la comparación TF-IDF vs. BM25 en una tabla de métricas.

1

Definir qrels — elegir 5 consultas y etiquetar a mano los documentos relevantes (graduado 0–3).

2

Implementar métricas — precision@k, recall, MRR, average precision y nDCG, desde cero.

3

Evaluar ambos sistemas — correr TF-IDF y BM25 sobre las 5 consultas; promediar cada métrica.

4

Defender una decisión — ¿qué sistema y qué (k_1, b) eligen para producción, y con qué métrica lo justifican?

Sus qrels reflejan SU criterio de relevancia: si marcaron d02 como relevante para “agua”, midieron también la falla de significado.

03

Descubrir sin consulta

Hasta ahora ustedes preguntan y el sistema responde. ¿Y si dejamos que el corpus revele su propia estructura?

- De retrieval a discovery
- K-Means sobre los vectores TF-IDF
- El mismo $\mathbb{R}^n$ de la Unidad 1
- Interpretar los grupos temáticos

Dos preguntas distintas sobre el mismo corpus



Retrieval (lo de hoy y la sesión pasada)

- Ustedes traen una consulta.
- El sistema ordena por relevancia.
- La pregunta la pone el usuario.



Discovery (lo nuevo)

- Nadie trae una consulta.
- El sistema agrupa lo que se parece.
- La estructura emerge del dato.

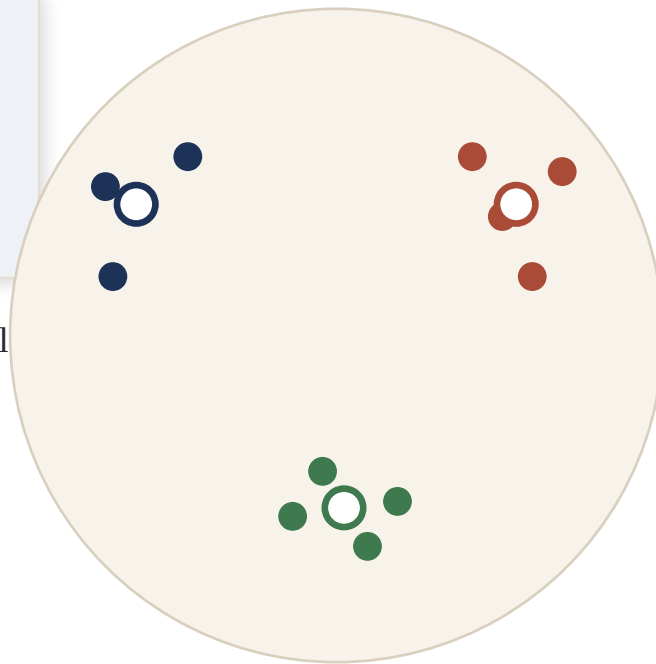
Esta es la filosofía de la Unidad 1 aplicada a texto: descubrir patrones sin hipótesis previa. Lo único que cambia es que los puntos ahora son documentos.

La idea: agrupar los vectores que ya construyeron

En el Lab 2 convirtieron cada documento en un vector TF-IDF. Esos vectores viven en $\mathbb{R}^n$ — exactamente el espacio donde corría K-Means en la Unidad 1.

El plan en una línea: reutilicen su K-Means de la Unidad 1, denle de comer la matriz documento $\times$ término, y lean qué grupos forma.

documento $\rightarrow$ vector TF-IDF $\rightarrow$ punto en $\mathbb{R}^n \rightarrow$ K-Means



3 grupos, 3 centroides

K-Means sobre texto: tres cuidados



Alta dimensión y dispersión

El vocabulario da miles de dimensiones, casi todas cero. Las distancias se vuelven menos informativas (maldición de la dimensionalidad).



Normalicen los vectores

K-Means usa distancia euclidiana, sensible a longitud. Normalicen a norma 1 (L2) para que euclidiana $\approx$ coseno — la métrica correcta para texto, como vimos.



Elegir k no es trivial

¿Cuántos temas hay? Usen el método del codo (inercia) y el coeficiente de silueta para guiar la elección — no adivinen.

Qué grupos emergen del corpus chiapaneco

Con $k=4$, K-Means sobre los vectores TF-IDF separa el corpus por co-ocurrencia de términos. Cada grupo se interpreta por sus términos de mayor peso en el centroide.

Grupo A • Clima y sequía

Términos top: sequía, lluvia, inundación, cultivo

Documentos: d01, d02, d04

Grupo B • Agro y exportación

Términos top: café, cacao, exportación, productor

Documentos: d03, d08, d12

Grupo C • Turismo y cultura

Términos top: turismo, visitante, destino, cultural

Documentos: d05, d09

Grupo D • Servicios y sociedad

Términos top: agua, laboratorio, robótica, sismo

Documentos: d06, d07, d10, d11, d13, d14

Lab 4 — Descubran los temas de su corpus



Reutilicen su K-Means de la Unidad 1 sobre la matriz TF-IDF del Lab 2. (sklearn permitido solo para validar.)

1

Construir la matriz — documento $\times$ término con pesos TF-IDF; normalizar cada fila a norma L2.

2

Elegir k — método del codo + silueta para $2 \leq k \leq 8$; justificar el k elegido con la gráfica.

3

Agrupar e interpretar — correr K-Means; para cada grupo, listar los términos de mayor peso del centroide y ponerle nombre.

4

Evaluar el descubrimiento — ¿los grupos tienen sentido temático? ¿qué documento quedó mal agrupado y por qué?

El paso 4 alimenta el cierre: busquen específicamente dónde el agrupamiento por palabras traiciona el significado.

¿En qué grupo cayó “crisis hídrica”?

Lo que esperaría un humano

d02 (crisis hídrica) junto a **d13** (agua potable): ambos hablan de problemas de agua.

Lo que hizo K-Means

d02 cayó con **d04** (sequía/maíz) por compartir la palabra “sequía” — no con **d13**, que dice “agua” pero no “hídrica”.

La misma falla, ahora en discovery

BM25 mejoró el ranking. K-Means reveló temas. Pero ambos siguen agrupando por palabras compartidas, no por significado. “Agua” e “hídrico” son cadenas distintas tanto para buscar como para agrupar.

Semana 7 — Embeddings: vectores donde “agua” e “hídrico” quedan cerca. Ahí el buscador Y el clustering por fin verán significado.

Lo que sigue

Próxima sesión

- Word embeddings: de palabras-símbolo a palabras-vector (word2vec, GloVe, FastText).
- Por qué “agua” e “hídrico” terminan cerca en el espacio.
- Embeddings de documento y búsqueda semántica: el arreglo a la falla de hoy.

Para antes de la próxima clase

- Terminar Labs 3, 3b y 4 y subirlos al repo (con AI_USAGE.md).
- Lectura: Manning et al., caps. 8 (evaluación) y 16 (clustering).
- Traer el documento peor agrupado del Lab 4: abriremos con él.

Recorrido de hoy en IR: de 1972 (TF-IDF) a 1994 (BM25), y el primer paso hacia el descubrimiento no supervisado de texto.

Bibliografía de la sesión

-  **Manning, C., Raghavan, P. & Schütze, H. (2008).** Introduction to Information Retrieval. Cambridge UP. — Cap. 8 (evaluación), 11 (modelo probabilístico/BM25), 16 (clustering). Gratuito en nlp.stanford.edu/IR-book.
-  **Robertson, S. & Zaragoza, H. (2009).** The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in IR, 3(4).
-  **Aggarwal, C. C. (2015).** Data Mining: The Textbook. Springer. — Cap. 13 (Mining Text Data): clustering y evaluación de texto.
-  **Han, J., Pei, J. & Tong, H. (2023).** Data Mining: Concepts and Techniques (4ª ed.). Morgan Kaufmann. — Cap. 10 (Cluster Analysis). Texto oficial del curso.
-  **Järvelin, K. & Kekäläinen, J. (2002).** Cumulated gain-based evaluation of IR techniques (nDCG). ACM TOIS, 20(4).